

Security Architecture

Project: Aegivex AI (OKX.AI Genesis Hackathon)

Version: 1.0

Document Type: Security Architecture

Prepared For: Development Team

Prepared By: Aegivex AI Team

1. Overview

This document defines the security architecture of Aegivex AI. The primary objective is to protect user accounts, AI services, APIs, databases, and blockchain interactions while ensuring confidentiality, integrity, and availability across the platform.

2. Security Objectives

The platform must adhere to the following key security objectives:

- Protect user data and privacy
- Secure API communication channels
- Prevent unauthorized access and privilege escalation
- Protect AI services and infrastructure from abuse
- Secure blockchain and smart contract interactions
- Detect and mitigate suspicious activity in real time
- Maintain comprehensive, tamper-evident audit trails

3. Architecture Layers

Security controls are applied sequentially across all operational layers:

Client Layer → Authentication Layer → API Gateway → Backend Security → AI Security Layer → Database Security → Infrastructure Security

4. Authentication & Authorization

Authentication Method

- **Mechanism:** JSON Web Token (JWT)
- **Core Features:** Secure Login, Access Tokens, Automatic Token Expiration, Session Validation, Secure Password Reset workflow
- **Password Storage:** Salted bcrypt hashing algorithm

Role-Based Access Control (RBAC)

Every incoming API request must verify user permissions prior to processing.

Role	Permissions
User	Read, Scan
Developer	Read, Write, Scan, View Logs
Admin	Read, Write, Scan, Manage Users, View Logs, Manage Settings

5. API Security

All API endpoints must enforce the following security practices:

- Mandatory HTTPS/TLS encryption
- JWT Authentication on protected routes
- Strict input validation and request sanitization
- Rate limiting to prevent abuse and denial of service
- Cross-Origin Resource Sharing (CORS) protection
- Secure HTTP response headers
- API versioning discipline
- Appropriate HTTP status codes for error handling without leaking stack traces

6. AI Layer Security

To safeguard artificial intelligence components, the platform must:

- Never expose API keys or internal credentials
- Never reveal system or internal instructions/prompts
- Prevent prompt injection and jailbreak attacks
- Validate and sanitize inputs before reaching the AI models
- Validate model outputs before serving them to users
- Prevent unsafe or malicious financial/technical recommendations
- Log AI processing failures and anomalous queries
- Enforce strict token usage limits per account/session

7. Database & Infrastructure Security

Database Security (PostgreSQL)

- Encrypt data in transit via SSL/TLS
- Store hashed passwords using bcrypt
- Restrict database user permissions based on least privilege
- Utilize UUIDs for primary keys to prevent enumeration
- Prevent SQL Injection through Object-Relational Mapping (ORM) and parameterized queries
- Encrypt sensitive user data at rest where required
- Automated backups and Point-in-Time Recovery (PITR)

Infrastructure Security

- Automated SSL Certificate management
- Firewall rules and Web Application Firewall (WAF) integration
- DDoS Protection at the network and application layers
- Secure environment variable management for production deployment
- High Availability configuration and continuous infrastructure monitoring

8. Input Validation & Threat Protection

Input Validation Targets

All inputs must be checked against strict schemas and regex patterns before processing:

- Wallet Addresses & Smart Contract Addresses
- Token Contract Addresses & Transaction Hashes
- Website URLs, Email Addresses, and Passwords

Threat Defense Scope

The system actively defends against:

- SQL Injection (SQLi) & Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF) & Replay Attacks
- Brute Force Login & API Abuse
- Prompt Injection & Phishing Attempts

9. Logging, Monitoring & Incident Response

Logging Scope

Log all critical security events excluding sensitive user credentials and private keys:

- User Login attempts (Successful and Failed)
- API Requests & AI Model Processing Requests
- Security Scan Activities, Security Events, and System Errors

Incident Response Lifecycle

1. **Detect:** Real-time identification of anomalous behavior or security breach
2. **Log:** Record detailed incident context safely
3. **Notify:** Alert system administrators immediately
4. **Isolate:** Quarantine affected services or lock compromised endpoints
5. **Investigate:** Determine root cause and scope of impact
6. **Remediate & Restore:** Apply patches, verify stability, and resume operations

10. Security Checklist & Compliance

Core Compliance Principles

- Least Privilege Principle & Secure by Default
- Privacy by Design & Zero Trust Architecture
- Defense in Depth & Secure Coding Practices

Pre-Release Verification Checklist

- ☐ HTTPS enforced across all environments
- ☐ JWT authentication and session validation functioning properly
- ☐ Password hashing with bcrypt verified
- ☐ Environment variables and API keys secured
- ☐ Rate limiting configured on API Gateway
- ☐ Database backups active and verified
- ☐ AI prompt protections and input/output filters active
- ☐ Centralized logging operational
- ☐ Vulnerability scanning passed with zero critical flaws

11. Future Enhancements & Expected Outcome

Planned Enhancements: Multi-Factor Authentication (MFA), Hardware Security Key Support, AI-Based Threat Detection, Real-Time Fraud Monitoring, Advanced WAF Integration, and Security Analytics Dashboard.

Expected Outcome: This security architecture ensures that Aegivex AI maintains a secure, resilient, and trustworthy ecosystem for Web3 users, minimizing operational risks while delivering high availability and user confidence.